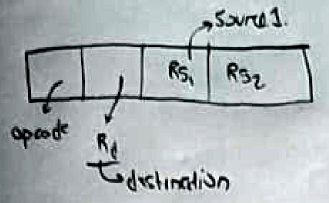
Assignment-1 (Chipguard)

x-16 (Processor) (Company already built register & memory
→ Design Binary format of instructions.



16 Registers → RO-R15 (Each register → 16 bits)

Data memory → 0-255

Inst. memory → 0-255

Instruction length → 16-24 bits.

Signed numbers → two's comp.

Instruction validity → Each valid binary instruction must have only one meaning.

Main challeng: → each instruction must have same length
Add R2, R4, R7 → 3 register fields required
NOP → No operands

① Register field width → $2^4 = 16$ → 4 bits to identify any register ④

② Load immediate → instruction that puts number directly into register ④

immediate value → -8 to +7 → these 4 bits can be used to rep -8 to +7 in 2's comp.

③ Bits needed for memory offset →

→ Registers → Small fast storage locations
memory → stores data at numbered locations

LD (load) → transfer data from memory → register

ST (store) → stores from register → memory.

④ offset
↳ Effective address = Base address + offset ④
If R2=20 (Base add = 20)
LD, R1, [R2+4] ↳ offset = 4
effective add = 24
→ R1 में 24 add. का data load हो जाएगा|

-8 → +7 → offset → 4 bits

→ Effective address must be b/w 0 & 255 &

even
0, 2, 4 ... 254

④ Bits needed for branch displacement
↳ Branch can change flow of execution

BEQ → Branch if equal
BNE → Branch if not equal

Branch displacement → signed number added to the address of next instruction to calculate branch target.

⑤

Branch displacement range
-16 to +15

2^5 → 5 bits needed

⑥ Bits needed for JMP target
JMP 100 (Jump to address 100)

Jump target → 0-255

JMP target field = 8 bits

Ex Branch instruction

BEQ, R1, R2+6

$\underset{\text{opcode}}{4} + \underset{R1}{4} + \underset{R2}{4} + \underset{\text{Displacement}}{5}$ ⑰

→ 24 bits better

· 3 register operation → ADD, SUB, OR, XOR

opcode	Rd	RS₁	RS₂	Unused
(4)	4	4	4	8

· Immediate instructions

opcode	R_d	Immediate	Unused
4	4	4	12

memory instructions →

opcode	R_d	R_b → Base reg	Offset	Unused
4	4	4	4	8

Branch inst.

opcode	RS₁	RS2	Displacement	Unused
4	4	4	5	7

Opcode	R
4	4

- MOV → MOV R_1, R_2 ✓

Op code	Rd	Rs
4	4	4

- RET → Return RET R_s → jump to the byte address stored in R_s

Op code	Rs	unused
4	4	16

- NOP (No operation)

op code	unused
4	20

- assigning op code to each operation

ADD 0000
SUB "
AND
OR
XOR
INC "
DEC
MOV "
LDI
LD
ST
BZQ "
BNE
JMP "
RET
NOP -1111

Que ⑤ ① Prove that every format fits in 24 bits ✓✓
② Prove that each field can represent its required range ✓✓
③ Prove that decoder is unambigious
→ we defined 4 bit opcode to each operation → therefore decodeder can easily identify operation from 4 bits.

① Add R_1, R2, R3

0000 | 0001 | 0010 | 0011 | 0000 ↳ unused
opcode | R_d | R_{s_1} | R_{s_2} |

② Encode LD R2, [R5+4]

1001 | 0010 | 0101 | 0100 | ↳ unused
LD opcode | ↓ RD | ↓ Base Register | ↳ offset

Que ⑦ Compare an alternative design

16 bit	24 bit (our)
→ saves memory	→ all instructions have same length.
→ Branch format needs 17 bits (Dont fit)	→ enough space for 5 bit branch displacement & 8 bit jump target